SOFTOMATIC
TECHNOLOGY SERVICES

EXAMATE · SYSTEM DESIGN · ELITE + BATCH 1

Lesson 1 · What System Design Is

and a 5-step answer framework. Told through two stories: a college-fest canteen that must not run out of chai, and an Amazon candidate handed a one-line prompt, "design a rate limiter", with nothing else.

MODULE
System Design · Part A
Foundations

TIME
About 75 minutes
plus 45 minutes of practice problems

YOU NEED
100 to 150 DSA problems
No prior design knowledge.

YOUR ROUTE TODAY

01 A vague prompt at Amazon
02 Coding round vs design round
03 Requirements with numbers
04 The 5-step answer flow
05 **A rate limiter, end to end**

LESSON 1 AT A GLANCE

What you will be able to do

System design is deciding **what to build, how big, and why**, out loud, when nobody has told you exactly. This lesson gives you the vocabulary and a 5-step flow you will reuse in every later lesson.

Explain the round
What a design round is, and how it differs from a coding round, in one minute.

LLD or HLD?
Tell a class-design prompt from an architecture prompt, and know what to do first in each.

Write the scope box
Turn a one-line prompt into features in, features out, scale and numeric targets.

Run 5 steps in 45 to 60 min
Clarify, estimate, data and API, high-level design, deep dive.

Availability maths
Compute downtime from nines, in series and in parallel, and use it to justify a choice.

Survive follow-ups
Answer "how do you approach this?" and three levels of follow-up.

Subject / module	System Design / Part A: Foundations
Difficulty	Beginner. No prior knowledge assumed.
Prerequisites	None. You have solved 100 to 150 DSA problems, and that is enough.
Next lesson	Lesson 2: Client-server, HTTP and API design basics
Companies covered	Amazon, Microsoft, Google, Cisco, Adobe, Atlassian, Uber, Flipkart

SOURCE-CONFIRMED
A dated first-person report naming the company and the question, Oct 2024 to Oct 2026, opened in full.

CANDIDATE-REPORTED
Compilations, aggregators, older than 24 months, undated, or text seen only in search extracts.

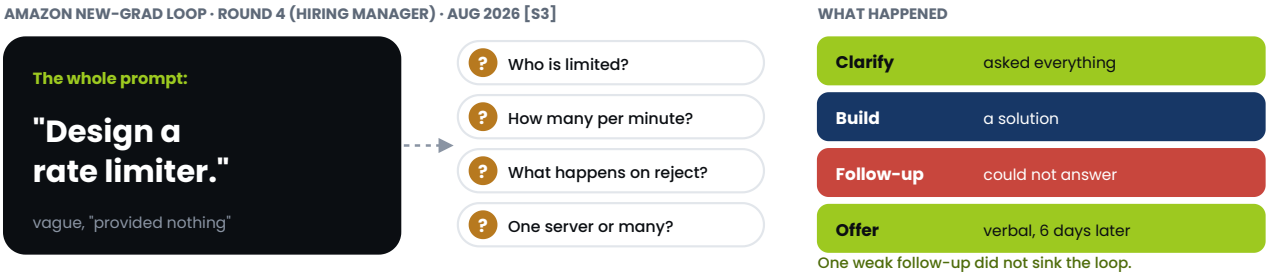
PATTERN-ONLY
Commonly asked, but no company source, so no company name is attached.

Practice problems and quiz questions are **original** and carry no label. Source IDs such as **[S3]** point to the Sources page at the end.

ONE LINE, NOTHING ELSE

"Design a rate limiter."

An Amazon new-grad candidate described a four-round onsite in a report published in September 2026 (interview in August 2026) [S3].



A rate limiter is a guard that rejects a client's requests once that client sends too many. Our running example for the whole lesson.

In the fourth round the hiring manager asked for a rate limiter. The report says the prompt "was vague and provided nothing" and the candidate "had to clarify everything" alone [S3]. They asked questions, built a solution, and then got a follow-up they could not answer. They still received a verbal offer six days after the onsite began [S3].

PAUSE AND PREDICT

What do you think was being graded in that round: the final code, or something that happened **before** any code was written?

Answer: Hold that thought. Concept 4 reveals what we can and cannot know. The short version: the first five minutes can sink a round, even though most of the marks are earned later, in the deep dive.

WHY THE PROMPT IS SO EMPTY

The interviewer is not hiding the requirements to be rude. They are checking whether you will **build the wrong thing very confidently**, which is a real skill that real companies pay to avoid.

INTERVIEW MODE: SAY IT LIKE THIS

- In a design round the prompt is deliberately incomplete. My first job is to make it concrete by asking questions.
- Jumping straight to boxes or code is the most common way to lose the round.

BEFORE THE JARGON

Planning a college-fest canteen

Your fest is in three weeks. The principal says: "Set up the food stall. Make it work." Nobody lights a stove yet.

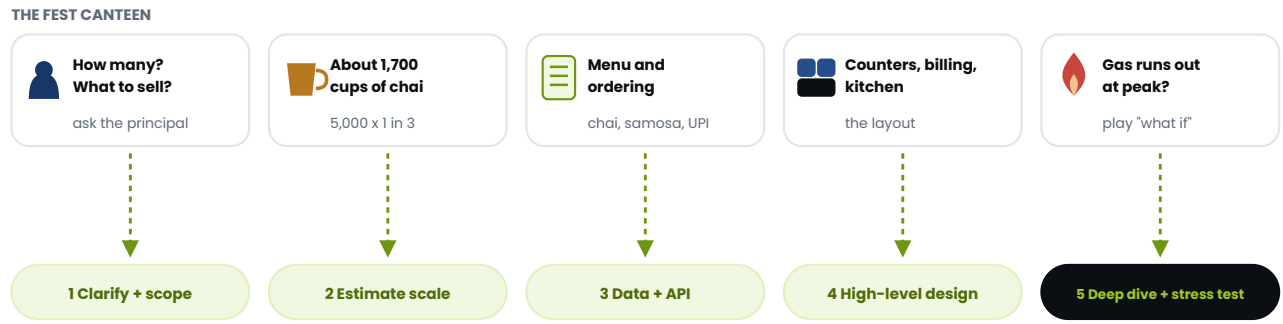
A sensible organiser asks first:

1. **How many people are coming?** 500 or 5,000? This is **scale**.
2. **What must the stall do?** Chai, samosas, a veg thali? Take UPI? These are **features**.
3. **How well?** No queue longer than 10 minutes; never run out of chai at peak. These are **quality targets**.
4. **What is out of scope?** No home delivery this year.

Then the numbers: 5,000 people, about 1 in 3 buys chai, so roughly **1,700 cups**. Then the menu and how people order. Then the layout: counters, a billing desk, a kitchen, a storeroom. Finally "what if": the gas cylinder ends at peak hour? The billing desk's UPI machine hangs?

THE MAPPING

The canteen is the **system**. The principal is the **interviewer**. The stove is **code**, and you are not touching it for a while.



Same order, two worlds. What, how many, how well, how it is laid out, and what breaks.

WHAT GOES WRONG

Where the analogy breaks. A second counter costs real money and time. In software you can start many copies of a program in minutes, but copies bring problems the canteen never has, such as two copies disagreeing about the same data. Later lessons deal with exactly that. (Also, nobody ever argued about "eventual consistency" over a samosa.)

TINY CHALLENGE

Pick any app on your phone (Swiggy, UPI, Zomato, a ticket app). Write two features it must have and one quality target, as a number. Answer: Example for a ticket app: "book a seat", "cancel a booking", "seat map loads within 2 seconds".

INTERVIEW MODE: SAY IT LIKE THIS

- A design answer follows the organiser's order: what, how many, how well, the layout, what breaks.
- I say that order aloud at the start so the interviewer can follow me.

EVERY WORD, ONE LINE

Vocabulary primer

Read it once; each term returns with an example. *Cache*, *queue*, *load balancer* and *database* are only named here; each gets its own lesson.

System
Programs and machines that together do a job for users ("book a train ticket").

Client
The program the user touches: a phone app or a browser.

Server
A program on a machine that waits for requests from clients and replies.

Request / response
A client asking something; the server's answer.

Component
One named part of a system: "the database", "the login service".

Requirement
What the system must do, or how well it must do it.

Functional req. (FR)
What it does, as a feature: "a user can post a message".

Non-functional req. (NFR)
How well: speed, uptime, safety of data.

Scope / descoping
Scope is what you will design; to descope is to say aloud what you will not.

Scale
How big the load is: users, requests, data.

DAU
Daily active users: different people who use it in one day.

QPS (RPS)
Queries (requests) per second hitting the system.

Latency
How long one request takes, send to answer. Unit: ms.

p99 latency
The time 99 of every 100 requests beat. p99 = 200 ms: 1 in 100 is slower.

Throughput
Work finished per unit of time, e.g. requests per second.

Availability
Fraction of time the system works, e.g. 99.9%.

Downtime
Time the system is not working.

Durability
Once it says "saved", the data is not lost, even if a machine dies.

Consistency
Whether everyone sees the same, latest data (Lesson 9).

API
The list of requests a system accepts and the answers it gives.

Endpoint
One request type in an API, e.g. "create a post".

Entity
A kind of thing the system stores: User, Order.

Architecture
Which components exist and how they talk to each other.

HLD
High-level design: box-and-arrow picture of a large system's components and data flow.

LLD
Low-level design: classes, relationships and methods for one program.

Machine coding
An LLD round where you write working code for a small system, timed.

Trade-off
Gaining one thing costs another (faster but less accurate).

Bottleneck
The part that limits the whole system's speed.

Deep dive
Going into two or three components in depth after the overview.

Stress test
Asking "what breaks?" about your own design: more traffic, a machine dying.

Fail open / closed
When a guard breaks: open lets everything through; closed blocks everything.

OA / HM round
Online assessment (timed test); hiring-manager round (projects, judgement, fit).

Whiteboard
The drawing surface: physical or a shared online canvas.

Rate limiter
A guard that counts each client's requests and rejects the extra ones.

API key
A secret string that identifies which client is calling.

HTTP 429 / counter
"Too Many Requests" reply code (Lesson 2); a counter goes up by one per request.

INTERVIEW MODE: SAY IT LIKE THIS

- Use the exact words: "functional requirements", "non-functional requirements", "scale", "p99 latency", "availability target". Interviewers listen for them.

CONCEPT 1 · WHAT A DESIGN ROUND IS, AND WHY IT EXISTS

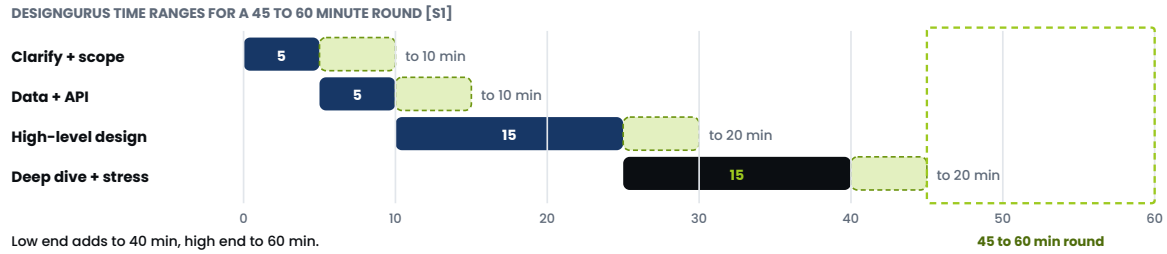
It tests how you decide what to build

A coding round shows whether you can solve a puzzle. A company also needs to know whether you can decide what to build when nobody has told you exactly, and explain why.

	CODING ROUND	DESIGN ROUND · 45 to 60 min
The prompt	Precise: input, output, constraints	Vague on purpose: "Design X"
The answer	One correct output	Many defensible designs
What you produce	Code that passes tests	Boxes, numbers and reasons
What is graded	Correctness, complexity	Scoping, trade-offs, depth
Your first move	Read, then code	Ask questions, then draw

Same candidate, two different exams. The design round grades your reasoning, not a single right answer.

A typical round runs **45 to 60 minutes** on one open prompt ("design a URL shortener"). You talk, draw and answer follow-ups. DesignGurus' 2026 guide lists the arc interviewers check, in order: did you **scope** the problem, **ground** it in data and APIs, **commit** to a defensible structure, and **go deep** without breaking down [S1].



Time budget [S1]. Solid = low end, dashed = high end. The ranges fit a 45 to 60 minute round.

WHAT GOES WRONG

Spending 20 minutes on clarifying leaves no time for the deep dive, where DesignGurus says most rubric points sit [S1]. Yet the same page calls **skipping** clarification "the most common loop-killer at every level" [S1]. Both are one prep site's opinion; no target company publishes a rubric that we found. Measure twice, cut once, except the saw is a whiteboard marker.

TINY CHALLENGE

Write your own minute-by-minute plan for a 45-minute round. Does it add up to exactly 45?
Answer: One option: 5 scope + 5 data/API + 17 design + 18 deep dive = 45.

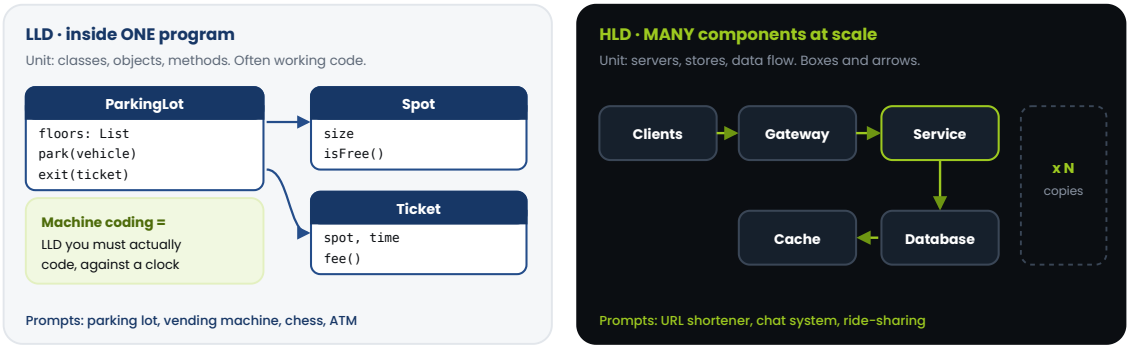
INTERVIEW MODE: SAY IT LIKE THIS

- "A design round tests how I decide what to build and why."
- "I will clarify, estimate, define data and API, draw the design, then go deep, and watch the clock so the deep dive is not skipped."

CONCEPT 2 · LLD, HLD AND MACHINE CODING

Which round am I in?

"Design X" can mean two very different exercises. Answer an HLD prompt with class diagrams, or an LLD prompt with load balancers, and you fail even if everything you say is true.



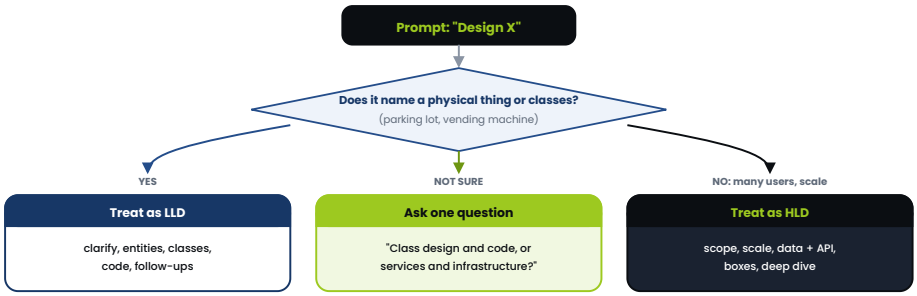
Different unit of design: classes versus components.

	LLD (LOW-LEVEL DESIGN)	HLD (HIGH-LEVEL DESIGN)
Unit of design	Classes, objects, methods	Components (servers, databases) and data flow
Output	Class structure, often working code	Box-and-arrow diagram plus reasoning
Typical prompts	Parking lot, vending machine, rate limiter (as code)	URL shortener, chat system, ride-sharing
What is graded	Clean structure, extensibility, edge cases	Requirements, scale reasoning, trade-offs

DesignGurus says plainly its framework is for distributed system design, and "no" for LLD or OOD rounds [S1]. LLD gets its own method in Lesson 15. A short LLD flow: **clarify, list entities, define classes and methods, write code, absorb follow-up requirements**. That sequence is our synthesis; the follow-up pattern is reported in an Amazon SDE-1 loop [S2].

MEMORY HOOK: LLD = LINES OF CODE, HLD = HUGE NUMBERS OF USERS

If the prompt names a **thing with rules** (a parking lot, a chess game), think classes. If it implies **millions of users**, think boxes. If unsure, ask.



When the prompt is unclear about the round type, one question beats a guess.

WHAT DO THE TARGET COMPANIES ASK FRESHERS?

Read the level column carefully

Three dated prompts first, then company by company. Much of the evidence is for SDE-2 or senior roles.

PROMPT	ROUND TYPE	EVIDENCE
"Low level design of a parking lot"	LLD	Amazon, Bengaluru, Mar 2025 SOURCE-CONFIRMED [S7]
"Design a URL shortener tool" (last of four rounds)	HLD	Amazon, Jun 2025 SOURCE-CONFIRMED [S9]
"Design a rate limiter"	Coding-style design, with follow-ups	Amazon, new grad, Aug 2026 SOURCE-CONFIRMED [S3]

COMPANY	WHAT WE FOUND	LEVEL	LABEL
Amazon	Rate limiter in the new-grad HM round, Aug 2026 [S3]. Parking-lot LLD [S7, S8]. An SDE-1 loop with an LLD round, page undated [S2]	New grad, SDE-1	S3, S7, S8 SOURCE-CONFIRMED S2 CANDIDATE-REPORTED
Microsoft	LLD (multi-floor parking lot, undo/redo) and HLD (distributed key-value store), 2026 [S5, S6]	SDE-2	SOURCE-CONFIRMED No fresher report found
Google	Blind threads say L3 (new grad) gets no system design [X1]	L3	CANDIDATE-REPORTED search extract
Cisco	Whiteboard of a system you built: QPS, rate limiting, circuit breaker, queues, sync vs async [S10]	Senior	SOURCE-CONFIRMED Jul 2026
Adobe	HLD required for MTS-2 and above; the OA reportedly has concept MCQs [S11]	Mid/senior (HLD)	CANDIDATE-REPORTED aggregator
Atlassian	Design round: requirements, API, database, list of services [S12]	SDE-2, undated	CANDIDATE-REPORTED
Uber	LLD or machine coding for interns and SDEs; HLD mid-level and up [X2]	Intern, SDE	CANDIDATE-REPORTED extracts; one post looks templated
Flipkart	Tier-3 SDE-1 loop (2024), no design round; HM round asked OOPs, DBMS, CN, OS [S4]	SDE-1	CANDIDATE-REPORTED loop 2024, posted 2026

TAKEAWAY FOR A FRESHER AIMING AT 15 TO 20 LPA

Expect **LLD more often than full HLD**, expect **concept questions inside other rounds**, and **prepare the HLD basics anyway**, because Amazon, Microsoft and Cisco reports show them appearing. The rate limiter itself gets both treatments later: Lesson 18 (LLD) and Lesson 25 (HLD).

WHAT GOES WRONG

A candidate hears "rate limiter", draws five servers and a database, and the interviewer wanted a class with a method `allow(clientId)`. Both are valid; only one matches the round.

INTERVIEW MODE: SAY IT LIKE THIS

- "Would you like the class design and code, or the service-level architecture?"
- "LLD is classes and methods; HLD is components and data flow."

TINY CHALLENGE

Classify: design a chess game, design WhatsApp, design an ATM. Answer: LLD, HLD, usually LLD (you could ask). See practice problem 2.

CONCEPT 3 · REQUIREMENTS: FUNCTIONAL AND NON-FUNCTIONAL

Good depends on what and how well

Without requirements you cannot tell a good design from a bad one. DesignGurus splits them into two kinds [S1].

Functional: what it does

Pick three to five core features. State them. Descope the rest. "A user can post a message."

Non-functional: how well it does it

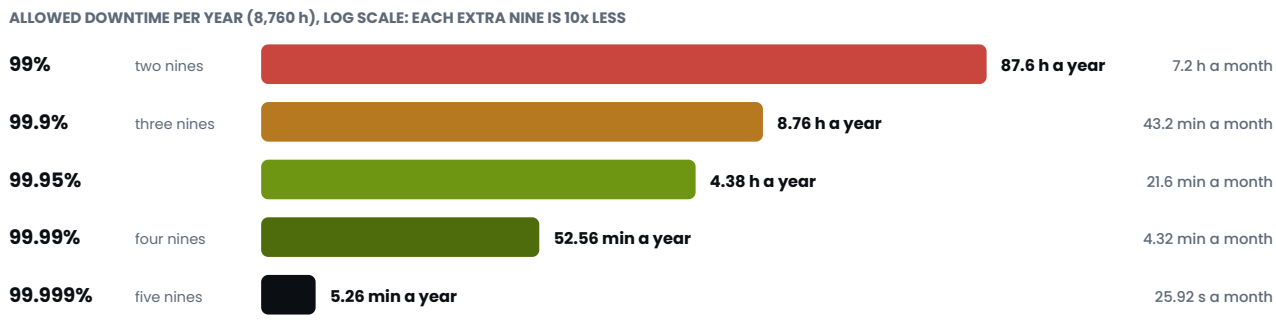
The four that matter most, per that guide: **availability, latency, consistency, durability** [S1]. Commit to a target for each, ideally a number.

Why numbers? A number forces a design choice. A label forces nothing.

VAGUE	CONCRETE	WHAT IT FORCES
"highly available"	"99.9% availability"	About 8.76 hours of downtime a year is allowed, so one single server is risky
"fast"	"p99 latency under 200 ms"	The slowest 1 request in 100 must still finish in 200 ms
"no data loss"	"orders are never lost once confirmed"	Data is written to more than one machine before saying "saved"

Derive it once, reuse forever. Availability = uptime / (uptime + downtime) [S13], so downtime fraction = 1 - availability. A year has 365 x 24 = **8,760 hours**; a 30-day month has 720.

AVAILABILITY	DOWNTIME FRACTION	PER YEAR	PER 30-DAY MONTH	PER DAY
99%	0.01	87.6 hours	7.2 hours	14.4 minutes
99.9%	0.001	8.76 hours	43.2 minutes	86.4 seconds
99.99%	0.0001	52.56 minutes	4.32 minutes	8.64 seconds

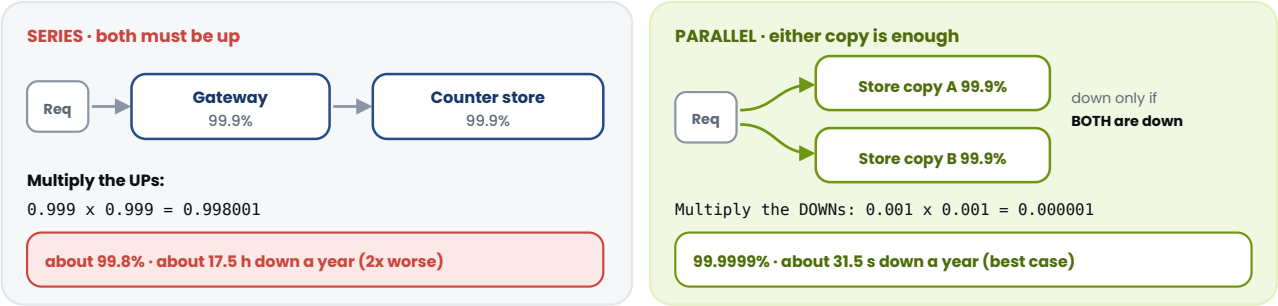


Check one cell: 0.001 x 8,760 h = 8.76 h. Each extra nine cuts allowed downtime by 10x.

AVAILABILITY OF A WHOLE PATH

Chains lower availability; copies raise it

Series rule. If a request must pass through two independent services, both must be up. Each at 99.9%: 0.999 x 0.999 = 0.998001, about **99.8%**. Downtime is about 0.001999 x 8,760 h = **17.5 hours a year**, double one service. This assumes failures are independent, which is an assumption, not a law.



Both assume failures are independent. Real copies share causes (same power, same bug), so treat parallel as a best case.

Parallel is previewed here; it returns in the deep dive (Concept 8) and in Lesson 11.

MEMORY HOOK: SERIES MULTIPLIES THE UPS, PARALLEL MULTIPLIES THE DOWNS

Two in a chain: 0.999 x 0.999. Two copies, either enough: 0.001 x 0.001. And for the nines: **three nines is about 9 hours a year, four nines under an hour**.

Running example: rate-limiter requirements

1. **Functional:** for each API key, allow at most 60 requests per minute; reject the rest with a clear "too many requests" reply.
2. **Non-functional (proposed):** the limiter check adds under 1 ms at p99 (our assumption, to confirm with the interviewer); 99.9% availability; decide fail open or fail closed when the limiter itself breaks.

WHAT GOES WRONG

Writing "scalable, available, fast" as labels. DesignGurus calls "we need it to be highly available" meaningless without a target [S1]. It is like telling the caterer "make enough food". Enough for whom, bhai?

INTERVIEW MODE: SAY IT LIKE THIS

- "Functional requirements are what it does; non-functional are how well it does it."
- "I will commit to numeric targets: availability, p99 latency, consistency level, durability."
- "99.9% availability is about 8.76 hours of downtime a year, about 43 minutes a month."

PAUSE AND PREDICT

A service is 99.99% available on its own. You put it behind a 99.9% gateway. Which number now limits you?

Answer: The gateway. 0.9999 x 0.999 is about 0.9989, worse than either alone, and below 99.9%.

TINY CHALLENGE

Convert "the app should rarely be down and quick" into two numeric targets. Answer: For example: 99.9% availability (about 8.76 h a year) and p99 latency under 200 ms.

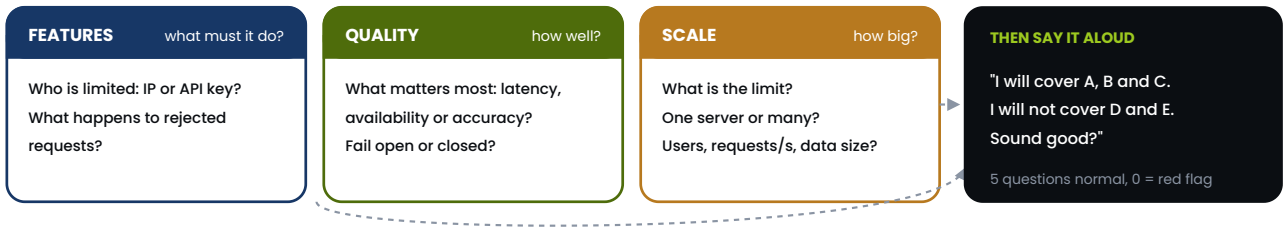
CONCEPT 4 · CLARIFYING AND SCOPING

Say your scope aloud, then check

The prompt is deliberately incomplete. Building the wrong thing with great skill is still a failure.

Ask in three groups (features, quality, scale), then **name three features, name what you will descope, and check in**: "Sound good?" [S1]. Five clarifying questions is normal, ten is fine, **zero is a red flag** [S1]. The Atlassian SDE-2 report gives the same order: functional and non-functional requirements, then API design with protocols, then database design, then the list of services [S12].

YOU ASK	INTERVIEWER SAYS (INVENTED FOR TEACHING)	YOU WRITE
Who is limited, an IP address or an API key?	API key	Limit key = API key
What is the limit?	60 requests per minute per key	Rule: 60 per 60 s
What happens to rejected requests?	Reply with an error	Reject with HTTP 429
One server or many?	Many servers behind one gateway	Distributed, so counters need care
Different limits for different customers?	Ignore that today	**Out of scope:** per-tier limits



Three groups, then one sentence of scope. The questions here are from the rate-limiter conversation above.

MEMORY HOOK: WHAT, HOW WELL, HOW BIG

Every clarifying question lands in one of three buckets. If all your questions are in the first bucket, you have not asked about **scale** yet, and scale is what shapes the design.

PAUSE AND PREDICT

The interviewer answers "many servers behind one gateway". Which later step does that one answer change the most?

Answer: The high-level design and deep dive: separate servers counting separately could let a client exceed the limit, so the counters need care (Concept 8).

CONCEPT 4 · WHAT STAYS ON THE BOARD

The requirements box

Four corners: what it does, how well, what you will not build, and the numbers. Everything you draw later must answer to this box.

FUNCTIONAL · what it does

- ✓ Per API key: at most 60 requests a minute
- ✓ Reject the rest with HTTP 429
- ✓ Clear "too many requests" reply

NON-FUNCTIONAL · how well

- ✓ Limiter check under 1 ms (p99)*
- ✓ 99.9% availability
- ✓ Decide: fail open or fail closed

OUT OF SCOPE · said aloud

- ✗ Per-customer tiers
- ✗ Dashboards
- ✗ Billing

NUMBERS · scale

- ✓ 1,000,000 registered keys
- ✓ 100,000 active in the peak minute
- ✓ about 100,000 checks per second

* my assumption, to confirm with the interviewer. The box stays on the board all interview; point at it when you justify a choice.

The requirements box you leave on the whiteboard. Functional, non-functional, out of scope, numbers.

BACK TO THE HOOK: WHAT WAS GRADED?

The candidate's own account says clarifying everything was required, and that the weak follow-up answers "didn't seem to have much impact", in their impression [S3]. That is **one candidate's guess, not a rubric**. What we can say safely: the prompt rewarded scoping, and one weak follow-up did not end that loop.

WHAT GOES WRONG

Two opposite mistakes. (a) Zero questions and drawing in 30 seconds, like ordering biryani for the whole fest without checking who is vegetarian. (b) Asking only about features and never about scale: "functional requirements without scale don't constrain the design enough" [S1].

INTERVIEW MODE: SAY IT LIKE THIS

- "I will cover A, B and C, not D and E. Is that acceptable?"
- "What scale: users, requests per second, data size?"
- "What matters most: latency, availability, or accuracy?"

TINY CHALLENGE

Write five clarifying questions for "design a notification service". Answer: Which channels? Who triggers them? How many a day? How fast must one arrive? May one be lost or sent twice?

CONCEPT 5 · ESTIMATING SCALE (A PREVIEW)

A number tells you which world you are in

The right design for 10 requests per second is wrong for 100,000. Lesson 4 gives the full toolkit; here is enough to start.

Round generously, state assumptions, aim for orders of magnitude [S1]. One conversion: a day has 24 x 60 x 60 = **86,400 seconds**, so **QPS = requests per day / 86,400**.

DesignGurus' example: 200 million DAU x 2 posts a day = 400 million writes a day, about 5,000 writes per second [S1]. Check: 400,000,000 / 86,400 = 4,630, which rounds to about 5,000.

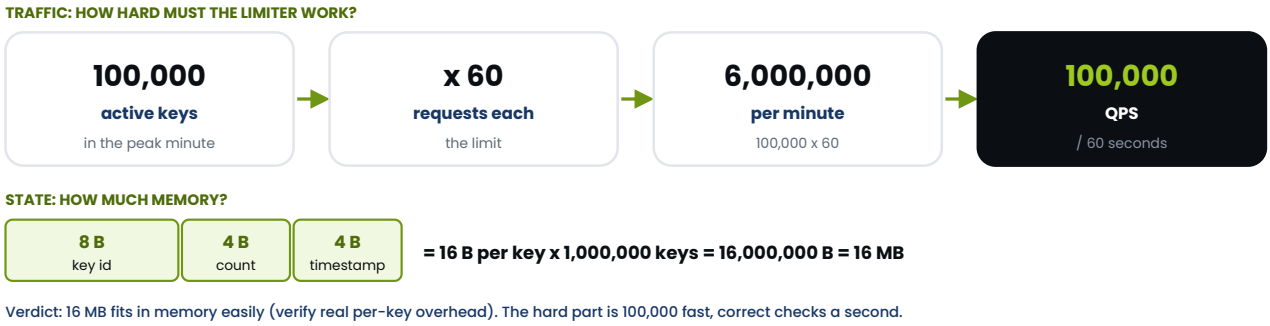
MEMORY HOOK: 86,400

A day is **86,400 seconds**; call it 100,000 for a quick check. One million a day is about 12 per second.

PAUSE AND PREDICT

10 million requests a day. Average QPS?

Answer: 10,000,000 / 86,400 = about **116 QPS**.



Rate-limiter estimate. Assumed: 1,000,000 registered keys, 100,000 active in the busiest minute, limit 60 per minute, 16 bytes per key.

What this tells you. 16 MB of state fits easily in memory, so storage is not the hard part. The hard part is doing **100,000 checks per second quickly and correctly across many servers**. Real stores add per-key overhead; verify before quoting 16 MB as final. And yes, you will divide by 86,400 in an interview, and no, nobody expects the last digit.

WHAT GOES WRONG

Estimating to the last digit (4,629 vs 5,000 QPS), or skipping estimation and later choosing a design that cannot carry the load. Cisco interviewers asked about QPS while a candidate drew their own system [S10], so the number is part of the conversation even for non-estimation prompts.

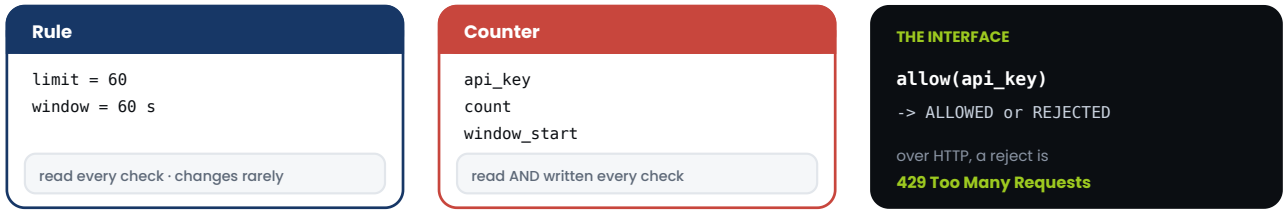
INTERVIEW MODE: SAY IT LIKE THIS

- "I will assume N users and M requests each a day, which gives about X QPS. Does that match what you have in mind?"
- "A day has 86,400 seconds, so I divide daily requests by about 86,400."

CONCEPT 6 · DATA AND API

Ground the boxes in data and requests

Diagrams that float with no data attached are easy to attack. List three or four entities (key fields only, with how they are accessed), then the four or five endpoints that matter [S1]. Not a full schema, not fifteen endpoints [S1].



The red header is the warning: about 100,000 writes a second at peak rules out slow storage before you draw a single box.

ENTITY	FIELDS	ACCESS PATTERN
Rule	limit = 60, window = 60 s	Read on every check; changes rarely
Counter	api_key, count, window_start	Read and written on every check (about 100,000 per second at peak)

```
# interface (conceptual, language-neutral)
allow(api_key) -> ALLOWED or REJECTED
# over HTTP, a rejected call gets status 429 (Too Many Requests)
```

WHAT GOES WRONG

Skipping access patterns. "Counters are written on every request" is one sentence that rules out slow storage later.

TINY CHALLENGE

List the entities and three endpoints for "design a to-do app". Answer: User, Task (id, title, done, due), maybe List; create task, list tasks, mark task done.

INTERVIEW MODE: SAY IT LIKE THIS

- "The main entities are X, Y and Z. Y is written on every request."
- "The core interface has N operations: ..."

MEMORY HOOK: NOUNS AND VERBS

Entities are the nouns the system remembers (Rule, Counter). **Endpoints are the verbs** clients call (allow). Write the access pattern next to every noun.

PAUSE AND PREDICT

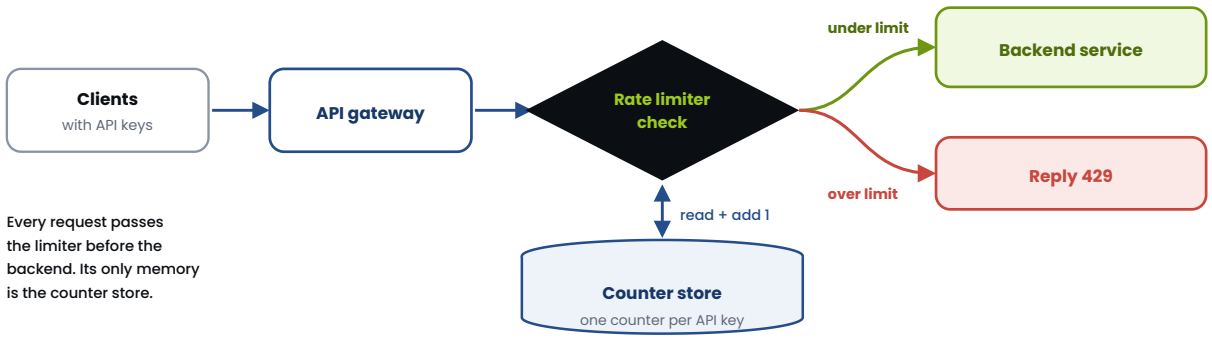
Which entity would hurt most if it lived on a slow disk, and why?

Answer: The Counter: it is read **and** written on every one of about 100,000 checks a second. The Rule is read often but almost never changes.

CONCEPT 7 · HIGH-LEVEL DESIGN

One picture a tech lead gets in 30 seconds

Boxes for components, arrows for data flow. DesignGurus' rule of thumb: **five to seven major components**, narrate every load-bearing choice, do not polish the drawing [S1]. A common start: a client, an entry point, the service, a data store.



Notice: every request passes the limiter before it reaches the backend, and the limiter's only memory is the counter store.

NARRATE THE LOAD-BEARING CHOICE

"The counters live in **one shared store**, not inside each server, because otherwise each server counts separately and a client can exceed the limit." Concept 8 shows the numbers.

WHAT GOES WRONG

Fifteen unlabeled boxes. Or boxes without saying why. DesignGurus: "the diagram is half the signal; the narration is the other half" [S1].

INTERVIEW MODE: SAY IT LIKE THIS

- "I will split the design into the request path and the data path."
- "I chose X over Y because of Z, and the cost is W."

TINY CHALLENGE

Draw your own rate-limiter diagram with only four boxes. Answer: Client, gateway with the limiter check, counter store, backend.

PAUSE AND PREDICT

Why does the backend never talk to the counter store?

Answer: Because the limiter decides before forwarding: it reads and updates the counter, then either forwards the request or replies 429. The backend only ever sees allowed traffic.

MEMORY HOOK: 5 TO 7 BOXES, EVERY ARROW LABELLED

If you cannot say **why a box exists in one sentence**, it should not be on the board yet.

CONCEPT 8 · DEEP DIVE AND STRESS TEST

A diagram is cheap; how it breaks is the signal

Pick two or three components using three criteria, in order: most complexity, what breaks first at scale, and the interesting trade-off [S1].

1 What breaks first at 10x scale?

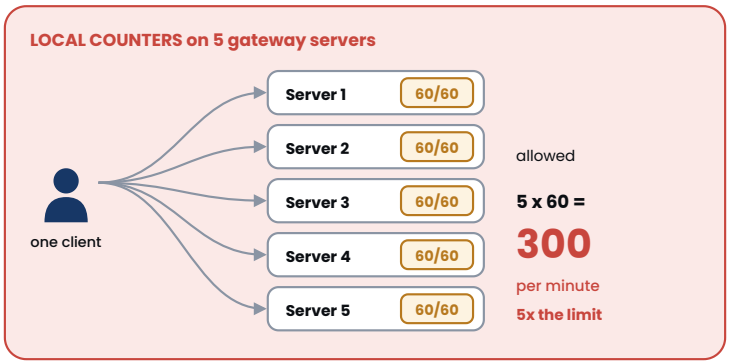
2 What is the cost driver, and what would halve it?

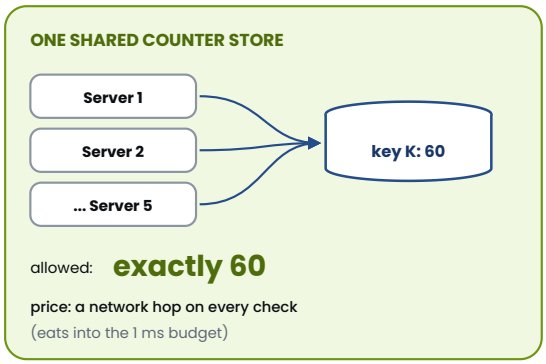
3 What does the on-call engineer see when it fails?

4 What is the consistency model, and which corner cases violate it?

Then stress-test: 10x traffic, a region or machine going dark, a cache filling up, cost, operations [S1].

Worked example 1: why a shared counter store. Assume 5 gateway servers, each with its own local counter, limit 60 per minute. If a client's requests spread over all 5, each server allows 60.





5 x 60 = 300 requests per minute, five times the limit. With one shared store the limit is exactly 60, at the price of a network hop per check.

PAUSE AND PREDICT

8 servers, each with local counters, limit 100 per minute. Worst case for one client?

Answer: 8 x 100 = **800 per minute**.

MEMORY HOOK: N SERVERS, N LIMITS

Local counters quietly **multiply the limit by the number of servers**: 5 x 60 = 300. One shared store keeps it at 60, and turns the store into the thing you must deep-dive next: its speed and its uptime.

INTERVIEW MODE: SAY IT LIKE THIS

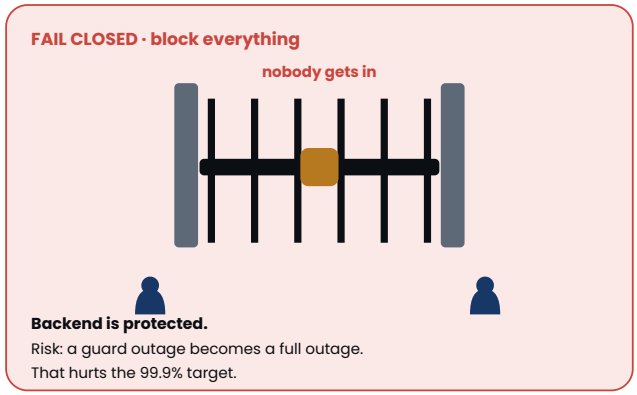
- "With local counters, a client spread over N servers gets N times the limit, so I keep one counter per key in a shared store and accept a network hop per check."

WHEN THE GUARD ITSELF BREAKS

Fail open or fail closed: pick one

Worked example 2. Assume the counter store is down for 5 minutes.





The college-gate question: leave it open when the guard faints, or lock everyone in?

CHOICE	WHAT USERS SEE	RISK
Fail open (allow everything)	Service keeps working	An abusive client gets unlimited requests for 5 minutes
Fail closed (block everything)	Every client gets rejected	A guard outage becomes a full outage, which hurts the 99.9% target

No single right answer. Pick one, say why, state the cost. DesignGurus: "hedging instead of committing" reads as evasion [S1].

Worked example 3: availability of the whole path. The counter store is in the path of every request. Gateway (99.9%) and store (99.9%) both required: about 99.8%, about 17.5 hours of downtime a year. Recover with **two copies of the store**: if failures are independent (assumption), both are down only 0.001 x 0.001 = 0.000001 of the time, which is **99.9999%**, about **31.5 seconds a year**. Real copies share causes of failure, so this is a best case.

WHAT GOES WRONG

Going wide instead of deep: touching ten components and unable to answer the four questions about any of them. Also skipping the stress test; DesignGurus calls designing without asking "what breaks?" a junior pattern [S1].

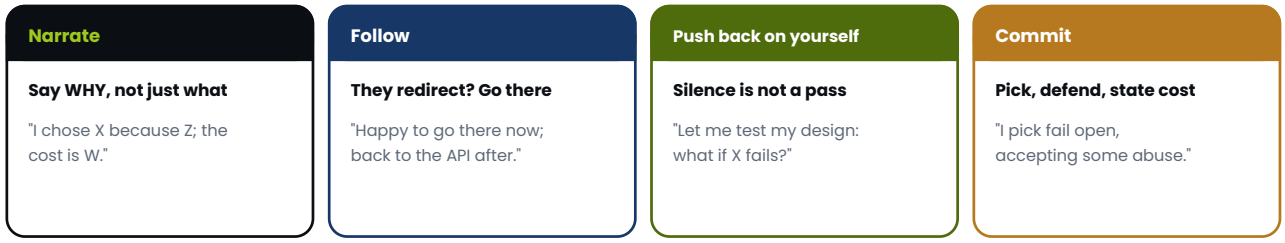
INTERVIEW MODE: SAY IT LIKE THIS

- "I will go deep on two things: the counter store and the failure behaviour."
- "At 10x traffic the first thing to break is X; I would fix it by Y."
- "If the limiter fails I would fail open/closed because ..., accepting ..."

CONCEPT 9 · HOW TO TALK: NARRATION, REDIRECTS, SILENCE

A design round is a conversation

The best diagram, narrated badly, scores badly.



Four moves. Narrate, follow, push back on yourself, commit.

1. **Narrate choices.** Say why, not just what.
2. **Follow a redirect.** The framework is scaffolding, not a script; if the interviewer wants a component now, engage [S1].
3. **Push back on yourself when they are silent.** Silence is not a pass; volunteer failure modes and cost [S1].
4. **Commit.** Pick an option, defend it, state the trade-off.
5. **Do not know a tool they mention?** Say so, and reason from what the tool must do. (Our advice, not a sourced rule.)

Worked example: a redirect. Minute 12, mid-way through data and API. The interviewer says, "Tell me about the counter store."

Weak: "Let me finish the API first."

Strong: "Sure. For the counter store I need fast reads and writes on a per-key number; here is how I would think about it, and I will come back to the API after."

WHAT GOES WRONG

Dogmatically insisting on finishing "step 3" first; DesignGurus calls this a flexibility failure [S1].

INTERVIEW MODE: SAY IT LIKE THIS

- "Happy to go there now; I will return to the API afterwards."
- "Let me test my own design: what happens if X fails?"

TINY CHALLENGE

Record yourself explaining the scope box for 90 seconds. Did you say a number? Answer: If not, add one: the limit, the QPS or the availability target.

MEMORY HOOK: NEVER FREEZE, PROBE, COMMIT

Narrate, **F**ollow, **P**ush back on yourself, **C**ommit. If the room goes quiet, that is your cue to attack your own design, not to wait.

EVERYTHING IN ONE PLACE

The 5-step answer flow

Alex Xu's book, as reported in secondary summaries, uses four steps: understand the problem and set scope, propose a high-level design and get buy-in, deep dive, wrap up [X3]. DesignGurus groups them as clarify and scope (with estimates), data and API, high-level design, deep dive [S1]. We split **estimation** out as its own step, because freshers forget it. (The book itself was not opened for this lesson.)



The dotted idea: the interviewer may jump you to any later step. Follow, then come back.

STEP	YOU SAY	YOU LEAVE ON THE BOARD	MINUTES (45-MIN ROUND)
1 Clarify and scope	"I will cover A, B, C, not D, E"	Scope box	4 to 6
2 Estimate scale	"About X QPS and Y GB"	Numbers	2 to 4
3 Data and API	"Entities are ..., endpoints are ..."	Entity list, 4 to 5 endpoints	5 to 8
4 High-level design	"The request path goes ..."	Boxes and arrows	12 to 15
5 Deep dive and stress test	"I will go deep on two things"	Failure and cost notes	12 to 15

Steps 1 and 2 together fit DesignGurus' 5 to 10 minutes for "clarify and scope"; the per-step split is our suggestion, not a sourced rule. The suggested minutes sum to **35 to 48**, so shrink step 4 or 5 if step 1 overruns.

MEMORY HOOK: "CHAI EVERY DAY HELPS DEVELOPERS"

Clarify, **E**stimate, **D**ata and API, **H**igh-level design, **D**eep dive. Same order as the canteen: what, how many, the menu, the layout, what if the gas runs out.

PAUSE AND PREDICT

Why does estimation come before data and API?

Answer: Because scale decides what the entities must survive: "written 100,000 times a second" changes the store you pick in step 4.

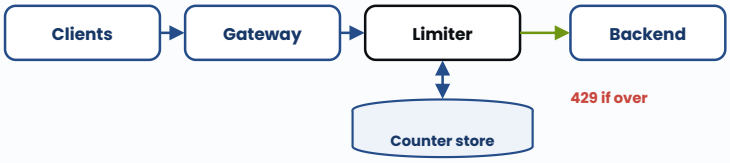
ONE WHITEBOARD, EVERY NUMBER TRACED

The rate limiter, end to end

SCOPE / SCALE / TARGETS
stays visible all interview

IN: per-key, 60/min, 429
OUT: tiers, dashboards, billing
1M keys, 100k active
~100,000 QPS, 16 MB
< 1 ms p99, 99.9%

HIGH-LEVEL DIAGRAM (5 to 7 boxes)



ENTITIES + 4-5 ENDPOINTS

```
Rule: limit, window
Counter: api_key, count,
window_start
allow(api_key)
-> ALLOWED / REJECTED
```

DEEP-DIVE NOTES: failures, cost, trade-offs

- 5 local counters = 300/min, so share one store
- Store down 5 min: fail open or closed? pick one
- 2 x 99.9% in series = ~99.8%: add a store copy
- 10x (1,000,000 QPS): store request rate breaks first

The whiteboard layout most candidates use. Keep the scope box in one corner so you can point at it whenever you justify a choice [S1].

Step 1, scope. In: per-key limit, 60 requests per minute, reject with 429. Out: per-customer tiers, dashboards, billing.

LINE	CALCULATION	RESULT
Registered keys	assumed	1,000,000
Active in peak minute	assumed	100,000
Peak requests per minute	100,000 x 60	6,000,000
Peak QPS	6,000,000 / 60	100,000 per second
Counter memory	1,000,000 x 16 B	16 MB (verify overhead)

Step 3, data and API. Entities Rule and Counter; interface `allow(api_key)`; HTTP 429 on reject. **Step 4.** Gateway, limiter check, shared counter store, backend (Concept 7).

TINY CHALLENGE

Change one assumption: the limit becomes 600 per minute, still 100,000 active keys. New peak QPS?
Answer: 100,000 x 600 / 60 = 1,000,000 QPS, exactly the 10x case in step 5, so the shared store is now the first thing to break.

Step 5, deep dive

1. Local counters on 5 servers allow up to 300 per minute instead of 60: use a shared counter store.
2. Store down for 5 minutes: fail open or fail closed? Pick one and justify.
3. Two required 99.9% components give about 99.8%: plan a second copy of the store.
4. First to break at 10x (1,000,000 QPS): the shared store's request rate, since every check touches it.

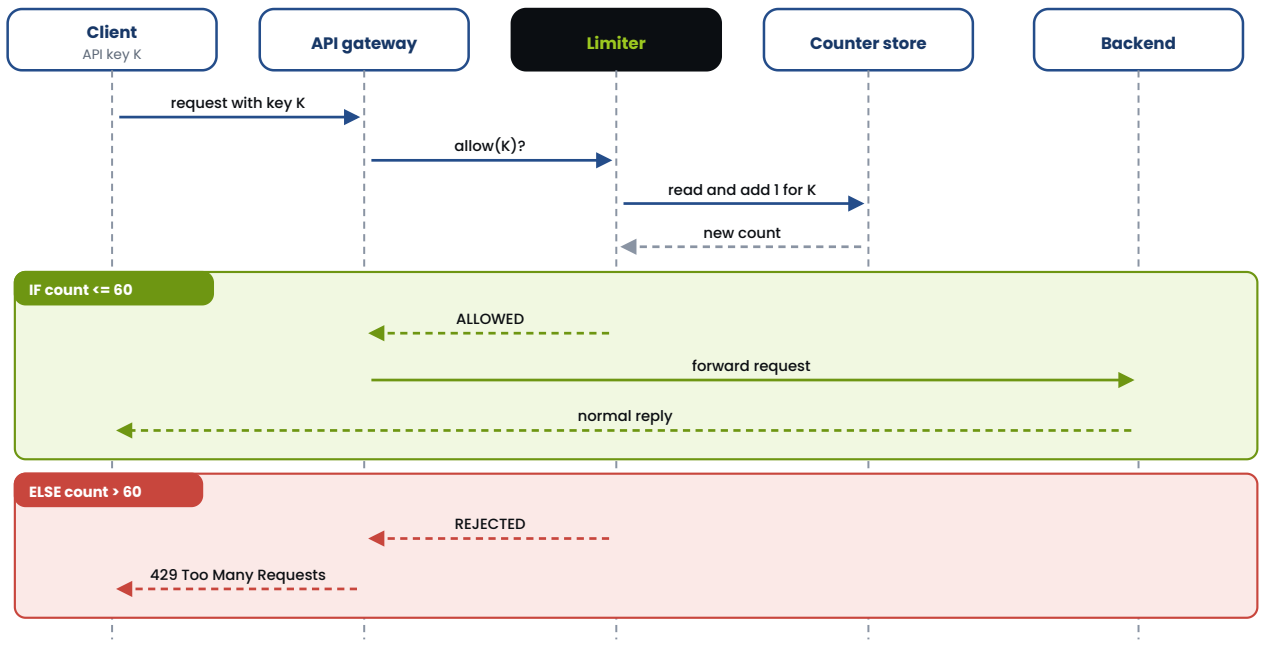
INTERVIEW MODE: SAY IT LIKE THIS

- "A gateway sends each request through a limiter that reads and updates one counter per API key in a shared store. Shared counters keep the limit across servers. The risk: the store becomes a bottleneck and a single point of failure."

WHEN DOES EACH CHECK HAPPEN?

The life of one request

You have met three diagrams: the LLD-or-HLD decision (Concept 2), the rate-limiter boxes (Concept 7) and the 5-step flow. Here is a fourth.



Notice: the counter store is touched on every request, which is why its speed and uptime matter so much.

Read it top to bottom

Time flows down. Solid arrows are calls, dashed arrows are replies. The green box runs when the count is within the limit, the red box otherwise.

Count the store calls

One per request, allowed or rejected. At 100,000 QPS the store sees 100,000 operations a second.

Interview use

Draw this when asked "walk me through one request". It shows where latency and failure can enter.

PAUSE AND PREDICT

The same key sends its 61st request inside one minute. Which arrows fire, and does the backend ever see it?

Answer: Request, allow(K)?, read and add 1, new count = 61, REJECTED, 429. The backend never sees it; the store still does, which is why it carries the load even for rejected traffic.

TINY CHALLENGE

Where would you add a 1 ms timer to check the latency target? Answer: Around the allow(K) call to the store and back: that round trip is the limiter's cost on every request.

THINGS THAT SOUND RIGHT AND ARE NOT

Common misconceptions

BELIEF	CORRECTION
"System design means memorising how Netflix or WhatsApp is built."	Memorised architectures show quickly and read as inflexibility; learn the building blocks and combine them [S1]. Like memorising a topper's answer sheet: great until the question changes.
"There is one right answer."	There is a defensible answer. The walkthrough in [S1] says there is no perfect answer; you are graded on reasoning and trade-offs.
"Start drawing immediately to look confident."	Zero clarifying questions is a red flag [S1]. Drawing in the first 30 seconds is the most common loop-killer in that guide.
"More boxes means a better design."	Five to seven major components is the target density; more makes the diagram unreadable [S1].
"Freshers never get system design."	Not in the reports we opened: an Amazon new-grad loop asked a rate limiter [S3]; an Amazon SDE-1 loop had an LLD round [S2]. But one Flipkart SDE-1 loop had none [S4], and Google L3 reportedly gets none [X1]. Prepare for the likely case: LLD plus basics.
"'Highly available' is a requirement."	It is a label. Write a number: 99.9% means about 8.76 hours of downtime a year.
"If the interviewer is silent, I am doing fine."	Silence is not a pass; fill it with your own stress-testing [S1].
"Saying 'it depends' is the safe answer."	Say what it depends on, then pick one and defend it. Repeated "it depends" reads as evasion [S1].

TINY CHALLENGE

Re-read the transcript in practice problem 6. Which of these eight beliefs did that candidate act on? **Answer: At least four: drawing immediately, more boxes is better, "highly available" as a requirement, and "it depends" as a safe answer.**

INTERVIEW MODE: SAY IT LIKE THIS

- "There is no single correct design. I will state assumptions, commit to a design, and explain the trade-offs I accepted."

TEAR OUT THIS PAGE

One-page cheat sheet

WHAT THEY ASK	THE ANSWER	THE TRAP
"Walk me through how you approach a design problem."	Clarify and scope, estimate, data and API, high-level design, deep dive and stress test	Reciting steps without applying them to the prompt
"Functional vs non-functional?"	What it does vs how well (availability, latency, consistency, durability)	Labels with no numbers
"LLD vs HLD?"	LLD: classes, methods, code. HLD: components and data flow at scale	Answering an LLD prompt with servers, or the reverse
"How much downtime is 99.9%?"	0.001 x 8,760 h = 8.76 hours a year; about 43.2 minutes a month	Mixing up 99.9% and 99% (87.6 hours a year)
"What is p99 latency?"	99 of every 100 requests are faster than this time	Saying it is the average
"Two services in a chain, each 99.9%. Total?"	About 99.8% (0.999 x 0.999)	Saying 99.9%; chains lower availability
"Why this design over another?"	Name the requirement it serves and the cost you accept	"Because it is popular"
"What if the guard component fails?"	Choose fail open or closed, say why, state the cost	Refusing to choose
"How would you spend 45 minutes?"	About 5 to 10 scope and scale, 5 to 10 data and API, 15 to 20 design, 15 to 20 deep dive (DesignGurus ranges)	Spending 20 minutes on step 1
"What if the interviewer jumps ahead?"	Follow them, return later	Insisting on finishing your own order

8.76 h
a year at 99.9%. 99% is 87.6 h; 99.99% is 52.56 min.

86,400
seconds in a day. Daily requests / 86,400 = average QPS.

C-E-D-H-D
Chai Every Day Helps Developers: the five steps in order.

WHAT YOU WILL SEE AT WORK

Where this shows up in real systems

1 · Design documents

Teams write a short document with goals, and often non-goals, before building. That is Concept 4 done on paper. (Common practice; we did not open a specific company template.)

2 · Rate-limit responses on real APIs

A rejected request typically gets status 429, and many APIs add a **Retry-After** header saying how long to wait. Illustrative output, not from a specific service:

```
$ curl -i https://api.example.com/v1/items
HTTP/1.1 429 Too Many Requests
Retry-After: 30
```

3 · Measuring latency yourself

This prints the total time of one request in seconds. Run it ten times and you get a feel for p50 versus the slow tail.

```
curl -w "%{time_total}\n" -o /dev/null \
     -s https://example.com
```

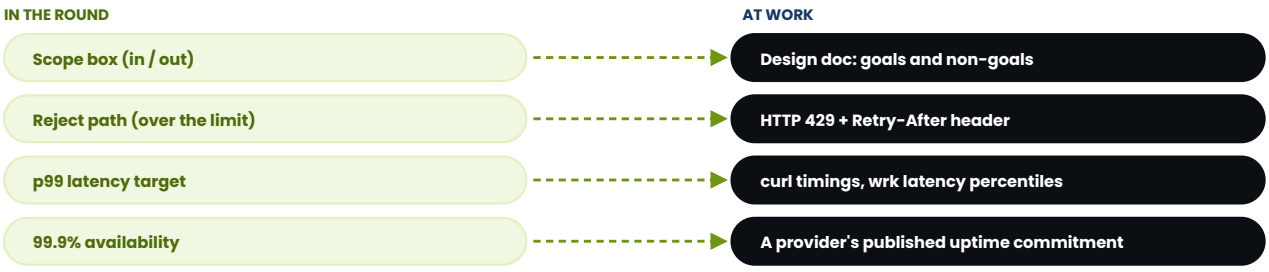
4 · Load-testing a design

A tool such as **wrk** sends many requests at once and reports requests per second and latency percentiles. Below: 4 threads, 100 open connections, 30 seconds. Verify flags against wrk's own documentation.

```
wrk -t4 -c100 -d30s http://localhost:8080/
```

5 · Availability promises

Cloud providers publish uptime percentages for their services; reading one turns "99.9%" from a number into a contract about monthly downtime. Check the current published figures for any service you cite.



Everything on your whiteboard has a twin at work. The round is a rehearsal for the job.

INTERVIEW MODE: SAY IT LIKE THIS

- "In real teams this first step is a written design doc with goals and non-goals."
- "Rate limits show up as HTTP 429 with a Retry-After header. Latency is measured in percentiles such as p99."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 1 and 2

Candidates mostly report the **design prompt**, not the process question, so few verbatim fundamentals exist for this lesson. Q3 to Q5 are pattern-only, and we say so plainly.

Q1 "Design a rate limiter" with a deliberately vague prompt SOURCE-CONFIRMED

Amazon, new-grad loop, round 4 (hiring manager), Aug 2026 [S3]. prachub.com/interview-experiences/amazon-new-grad-software-engineer-interview-experience-aws-offer-after-four-onsite-rounds

Model answer (spoken). "Before I design, let me clarify. Who is limited: IP address or API key? What is the limit and the time window? What should happen to rejected requests? One server or many? I will assume per API key, 60 requests a minute, reject with a 429, many servers, and leave out per-customer tiers. At about 100,000 requests a second at peak, I will use a shared counter store."

> **Why ask instead of picking a design?** "The right design depends on the answers. A per-IP limiter on one server and a per-key limiter on many servers are different problems. Guessing risks solving the wrong one."

> **Five servers keep local counters, limit 60. Most one client can send?** "5 x 60 = 300 per minute, five times the limit, if the client's requests reach all five."

> **The shared store goes down. What do you do?** "Choose fail open or fail closed. Open keeps the service up but lets abuse through; closed protects the backend but turns a guard outage into a full outage. For a public API with a 99.9% target I would lean towards fail open for a short outage and alert loudly, accepting some abuse, and confirm that preference with the interviewer."

Weak: "I'll use Redis and a sliding window."

Strong: "Let me confirm who is limited and what happens on rejection, then I will pick the counter approach."

Q2 "Why did you choose this data structure or design over the alternatives?" CANDIDATE-REPORTED

Microsoft, SDE-2 design rounds, Feb to Mar 2026. One report: rounds were about the "why" behind data-structure choices [S5]; another: Microsoft values explaining the reasoning behind technical choices [S6]. These describe the focus, not a verbatim question.

Model answer. "I chose a shared counter store over local counters because the requirement says the limit must hold across servers. Local counters are faster but allow up to N times the limit on N servers. I accept an extra network hop per request, which eats into my 1 ms budget, to get an accurate limit."

> **Why does accuracy matter more than speed here?** "The stated requirement is a hard limit. If approximate were fine, local counters with periodic syncing would win on speed."

> **How much time does the extra hop cost?** "I would assume a fraction of a millisecond to a few milliseconds inside one data centre, the main risk to a 1 ms p99 target. I would measure it, not guess." (Exact intra-data-centre latency is not certain; verify with measurements.)

> **When would you pick the other option?** "If traffic were small, or slightly exceeding the limit were harmless, local counters are simpler and add no new single point of failure."

Weak: "It is what everyone uses."

Strong: "It meets requirement R and costs C, which I accept because ..."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 3 to 5

Q3 "How do you approach a system design question? Walk me through your steps."

PATTERN-ONLY

No company found. A how-to-answer question appears in prep-site lists [S13], with no company attribution.

Model answer. "Five steps. One, clarify and scope: features in, features out, numeric targets. Two, estimate scale in QPS and storage. Three, define the main entities and four or five endpoints. Four, draw a high-level design of five to seven components, narrating why each is there. Five, go deep on two or three components and stress-test: what breaks at 10 times load, what if a machine fails, what it costs."

> **Why this order?** "Each step constrains the next. Scale decides whether one machine is enough; the data and API decide what the boxes must store and serve."

> **25 minutes in, still on the high-level design. Now what?** "Force a transition: wrap the design and pick two areas to go deep on, since the deep dive is where most points sit [S1]."

> **Does this order apply to a class-design question?** "No. For LLD I clarify, list entities, define classes and methods, code, then handle follow-ups. The five steps are for distributed design [S1]."

Weak: "I just start drawing and adjust."

Strong: "I scope first, state numbers, then design, then attack my own design."

Q4 "What are functional and non-functional requirements? Give examples." PATTERN-ONLY

No company found. Appears in beginner question lists with no company [S13].

Model answer. "Functional requirements are what the system does: a user can post a message. Non-functional are how well: 99.9% availability, p99 read latency under 200 ms, orders never lost once confirmed."

> **Why separate them?** "Functional ones decide which components exist; non-functional ones decide how those components are built and replicated."

> **What does 99.9% allow?** "0.001 x 8,760 hours = 8.76 hours of downtime a year, about 43 minutes a month."

> **How does a 99.99% target change the design?** "Allowed downtime drops to about 52 minutes a year, so a single machine or data centre is not enough; I need redundant copies and automatic failover."

Weak: "Non-functional means performance and stuff."

Strong: "Measurable quality targets, and each one forces a design decision."

Q5 "What is the difference between LLD and HLD?" PATTERN-ONLY

Not found as a verbatim question. An aggregator describes the two Adobe round types, LLD as object-oriented modelling and HLD as architecture [S11], but does not report this as a question.

Model answer. "LLD designs the inside of one program: classes, relationships, methods, often with code. HLD designs how many components and servers fit together for scale. A parking lot is LLD; a URL shortener for millions of users is HLD."

> **Why ask LLD from freshers?** "It shows clean code and extensibility without experience of large systems. In an Amazon SDE-1 report the candidate wrote that LLD is fair game even for SDE-1 [S2]."

> **Is a rate limiter LLD or HLD?** "Both exist. As code, one class with an allow(key) method. Across many servers, it needs a shared counter store. I would ask which one they want."

> **Can one design answer both?** "Start with the class that holds the logic, then show how it runs on many servers. That bridges LLD to HLD."

Weak: "HLD is bigger."

Strong: "Different unit of design: classes versus components."

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 6 and 7

Q6 "Walk through a system you built", with probing follow-ups SOURCE-CONFIRMED

Cisco, onsite whiteboard, Jul 2026, Senior role [S10]. A senior question; no fresher version found, but project walk-throughs are also reported for freshers in HM rounds [S4].

Model answer (structure). "I will describe the problem it solved, the users and scale in numbers, the main components, and one decision I would change. For scale: about N requests a day, so about N / 86,400 QPS." *What was reported:* while the candidate drew, interviewers asked about QPS, rate limiting, circuit breaker, queues, sync vs async, and REST vs gRPC vs GraphQL [S10].

> **Why did you add a queue?** "To decouple the slow step from the user's request, so the reply is not delayed. The cost is a delay before the work completes."

> **What QPS did it handle; what was the bottleneck?** "I give the number with its basis, and name the first component that would break at 10 times load."

> **What would you do differently now?** "One concrete change, with the requirement that motivates it."

Weak: A list of technologies.

Strong: A problem, a number, a decision, a cost.

Q7 "What does the design round expect?" (an advice-style report) CANDIDATE-REPORTED

Atlassian, SDE-2, India. Page undated; year not verified [S12]. leetcode.com/discuss/interview-experience/1795183/atlassian-sde-2-remote-india

Model answer. "I would gather requirements, define the API and protocol, sketch the data model, and name each service, even if time runs out before a full diagram." *What was reported:* stay calm; they do not want a flawless solution; start with functional and non-functional requirements, then API design with protocols, then database design; you may not reach the diagram, so list all services. The example prompt was a tag management system [S12].

> **Why list services if there is no diagram?** "A list shows the whole design in your head even when time is short."

> **The interviewer keeps asking about one detail.** "Follow into it; the redirect is intentional [S1]."

> **Which protocol for the API, and why?** "State a protocol, give one reason and one cost. Lesson 2 covers the choices."

Weak: "I would draw boxes."

Strong: "Requirements, API, data, services, in that order, with a reason at each step."

THE PATTERN BEHIND ALL SEVEN

Every strong answer has the same shape: **a reason, a number, a decision, a cost**. Every weak one is a label or a technology name.



Weak answers stop at a label ("highly available") or a tool name ("Redis", "Cassandra because it is popular").

Q1, follow-up 2, in four pieces. Use the same four pieces for any follow-up.

HOW TO THINK, STEP BY STEP

Scenario questions 1 to 4

Labels refer to the underlying question or situation, not to the step lists, which are original.

1. You hear only "Design a URL shortener."

Amazon, interview Jun 2025, last of four rounds

SOURCE-CONFIRMED [S9]

1. Do not draw. Ask functional questions: create a short link, redirect, expiry, custom alias, analytics?
2. Ask non-functional: redirect latency, availability, can links ever be lost?
3. Ask scale. Say 100 million new links a month, 100 reads per write. A month is 30 x 86,400 = 2,592,000 s. Writes: about 39/s. Reads: about 3,860/s.
4. Say scope aloud: "Create and redirect in; analytics and custom aliases out. OK?"
5. Only now move to data and API.

Why this works: every later choice (cache, storage) is justified by the read-heavy 100:1 ratio you just computed.

2. 25 minutes in, and you only have a scope box.

PATTERN-ONLY

1. Say it: "I have used a lot of time on scope, so I will move quickly."
2. Draw the simplest request path in two minutes: client, entry point, service, store.
3. Pick the two riskiest components and announce them as your deep dive.
4. Keep the last 10 minutes for failure and cost.

Why this works: compress earlier steps and force the transition, since deep dives carry most points [S1].

3. The interviewer jumps to a component you have not designed.

PATTERN-ONLY

1. Follow immediately: "Happy to go there now."
2. State the assumption you need from the skipped steps in one sentence (for example, the scale).
3. Go deep with the four questions: 10x, cost, what on-call sees, consistency.
4. Offer to return: "I will come back to the API after."

4. Minute 35: you spot a flaw in your own design.

PATTERN-ONLY

1. Say it before they do: "I see a problem with my earlier choice."
2. Quantify: "With 5 servers and a limit of 60, a client can send 300 per minute."
3. Propose the fix, a shared counter store, and its cost: a network hop per request and a new single point of failure.
4. Ask whether to redraw or just note the change.

Why this works: you just debugged your own design live, which is literally the job. Designing without asking "what breaks?" is a junior pattern [S1].

HOW TO THINK, STEP BY STEP

Scenario questions 5 to 8

5. The interviewer says nothing and nods.

PATTERN-ONLY

1. Do not read silence as approval [S1].
2. Volunteer: "Let me stress-test this. At 10 times the traffic, the first thing to break is X."
3. Add one sentence on cost and one on operations (which alert fires).

6. "Design a parking lot." Which round is it?

Amazon Bengaluru, Mar 2025, "Low level design of parking lot" SOURCE-CONFIRMED [S7]; Amazon San Francisco, May 2025, parking system SOURCE-CONFIRMED [S8]; Amazon SDE-1 LLD round, undated CANDIDATE-REPORTED [S2]

1. Ask: "Classes and code, or the infrastructure?" The reports above all describe class-level designs.
2. For LLD: list entities (vehicle types, spots, tickets), then the main methods, then code.
3. Expect follow-up requirements. In [S2] the candidate got 5 to 6, and says a clean base design made each a one-line change.

7. The interviewer mentions a tool you do not know.

PATTERN-ONLY

1. Say so briefly: "I have not used that."
2. Describe what the thing must do in your own words (store, queue, cache).
3. Reason from the requirement and ask them to correct you.

8. You are a fresher; "walk me through your project".

Flipkart SDE-1 HM round, resume and projects; loop 2024, posted Mar 2026 CANDIDATE-REPORTED [S4]

1. Give the problem, the users, a number (even a small one), and the components.
2. Name one decision and the alternative you rejected, with a reason.
3. Say what breaks if usage grows 100 times. That shows design thinking even for a college project.

WHERE THE SCENARIOS STRIKE IN A 45-MINUTE ROUND



S7 (unknown tool) and S8 (project walk-through) can come at any minute.

Know the danger minutes. Minute 25 still on scope, minute 35 a flaw, the last ten minutes in silence.

THE PATTERN BEHIND EVERY SCENARIO

Name what is happening out loud, **quantify** it with one number, **choose** a move, and **state its cost**. Interviewers forgive a wrong turn; they do not forgive silence.

SOLVE THESE BEFORE YOU LOOK

Practice problems 1 to 3

Original problems, not reported questions. Cover the solution, try it on paper, then check.

P1 Functional or non-functional? EASY

Classify each: (a) users can upload a profile photo; (b) 95% of pages load within 2 seconds; (c) no confirmed order may be lost; (d) admins can export a CSV; (e) the system handles 10,000 requests per second at peak; (f) users get a notification when a seat frees up; (g) user data stays in India.

Hint. Is it a feature someone clicks, or a quality measured by a number or guarantee?

SOLUTION

(a) functional. (b) non-functional: latency, a p95 target. (c) non-functional: durability. (d) functional. (e) non-functional: scale or throughput. (f) functional. (g) non-functional: a constraint on where data lives (some call it a compliance or data-residency constraint).

P2 Classify the prompt EASY

Say LLD, HLD, or "ask": (i) design a chess game; (ii) design WhatsApp; (iii) design an ATM; (iv) design a rate limiter.

Hint. Does it name classes and rules, or millions of users?

SOLUTION

(i) LLD. (ii) HLD. (iii) usually LLD (state and classes); you could ask. (iv) ask: as code it is LLD, across many servers it is HLD.

P3 Downtime NUMERICAL

(a) Downtime per year at 99.95%? (b) Two independent services in a chain, 99.95% each: combined availability and yearly downtime? (c) Two independent copies of a 99.9% service, either enough: availability and yearly downtime in seconds?

Hint. Downtime fraction = 1 - availability. Series: multiply availabilities. Parallel: multiply the downtime fractions.

SOLUTION

(a) 0.0005 x 8,760 h = 4.38 hours. (b) 0.9995 x 0.9995 = 0.99900025, about 99.9%; downtime about 0.00099975 x 8,760 = 8.76 hours. (c) 0.001 x 0.001 = 0.000001, so 99.9999%; downtime 0.000001 x 8,760 h x 3,600 s/h = 31.536 seconds a year, assuming independent failures, which real copies often are not.

HOW TO CHECK YOUR OWN ANSWER

Every requirement has a number. Every number has its assumption and unit. Every box has a reason. Every choice has a cost. If any of the four is missing, the answer is incomplete, even if the design is right.

SOLVE THESE BEFORE YOU LOOK

Practice problems 4 to 6

P4 URL shortener scale NUMERICAL

100 million new URLs a month, reads 100x writes, 30-day month, 500 bytes per stored URL, peak 2x average (assumption). Average and peak QPS for writes and reads, and storage after 5 years?

Hint. Seconds in 30 days = 30 x 86,400. Storage = URLs x bytes.

SOLUTION

Seconds: 2,592,000. Average writes: 100,000,000 / 2,592,000 = 38.6, about 39/s. Average reads: 38.6 x 100 = about 3,860/s. Peak (2x): about 77 writes and 7,720 reads per second. Storage: 100,000,000 x 500 B = 50 GB a month; x 60 months = 3,000 GB = **3 TB** (ignoring replication and overhead, which multiply it).

P5 Write the scope box MEDIUM

Interviewer: "Design a chat app." Invent the clarifying answers and write a scope box with in-scope, out-of-scope, scale and three numeric targets.

Hint. Pick three features, name three exclusions, give DAU, messages per user per day and a latency target.

SOLUTION

One valid answer. **In:** one-to-one messages, delivery status (sent/delivered), message history. **Out:** group chat, voice or video calls, end-to-end encryption design. **Scale (assumed):** 10 million DAU x 40 messages = 400,000,000 a day / 86,400 = about 4,630 messages per second. **Targets:** delivered to an online recipient within 500 ms (p99); 99.9% availability (about 8.76 h a year); no message lost once marked "sent".

P6 Spot the mistakes HARD

Read the transcript and list every mistake. Warning: it hurts, like a group-project presentation where nobody read the topic.

```
"Design a notification service? Okay, I'll start drawing. [draws 14 boxes with no labels] We need it to be
highly available and scalable. We will support email, SMS, push, in-app, WhatsApp, voice calls,
scheduling, templates, analytics, A/B tests, and user preferences. I'll use Cassandra because it's
popular. It depends on the use case. It depends on traffic. [aligns the boxes for four minutes] ... and
that's the architecture."
```

Hint. At least seven problems. Think scope, targets, scale, justification, hedging, polish and the stress test.

SOLUTION

(1) No clarifying questions; drew immediately. (2) Fourteen unlabeled boxes. (3) "Highly available and scalable" has no numbers. (4) Eleven features, no scoping or descoping. (5) No scale estimate. (6) Technology chosen with no reason ("because it's popular"). (7) Repeated "it depends" without picking. (8) Four minutes polishing the diagram. (9) No deep dive, failure modes, cost or operations. Each maps to a mistake DesignGurus lists [S1].

TEN QUESTIONS, TWO MINUTES EACH

Mini quiz

Original questions. Cover the answers, answer all ten, then check.

1. After a vague prompt, the first thing to do is:
A) draw boxes B) ask clarifying questions and set scope C) choose a database D) estimate the cost
Answer B [S1]. A is the named loop-killer; C is premature before requirements; D comes later, in the stress test.

2. Which is a non-functional requirement?
A) a user can post a message B) p99 read latency under 200 ms C) a user can follow another user D) an admin can delete spam
Answer B. It measures how well; A, C and D are features.

3. About how much downtime a year does 99.9% allow?
A) 8.76 hours B) 52.56 minutes C) 87.6 hours D) 4.38 hours
Answer A. 0.001 x 8,760 h. B is 99.99%, C is 99%, D is 99.95%.

4. Two independent services in a chain, each 99.9%. Combined?
A) 99.9% B) 99.8% C) 99.99% D) 100%
Answer B. 0.999 x 0.999 = 0.998. A ignores the second service; chaining never improves availability.

5. Which order matches the 5-step flow?
A) HLD, scope, estimate, data/API, deep dive B) scope, estimate, data/API, HLD, deep dive C) deep dive, scope, HLD, estimate, data/API D) estimate, HLD, scope, deep dive, data/API
Answer B. Each step constrains the next; the others put design before requirements or numbers.

6. The interviewer jumps ahead before you finish. Best response?
A) refuse and finish first B) follow them and return later C) stay silent D) restart from scope
Answer B [S1]. Scaffolding, not a script. A is inflexible, C wastes the chance, D burns time.

7. Which best describes LLD?
A) choosing regions and load balancers B) designing classes, relationships and methods C) estimating storage D) choosing a CDN
Answer B. A and D are HLD infrastructure choices; C is an estimation step of HLD.

8. Why state non-functional requirements as numbers?
A) it looks professional B) numbers force design choices, such as ruling out a single server C) interviewers forbid labels D) databases need them
Answer B. 99.9% allows only about 8.76 h a year. A is cosmetic, C is false, D is irrelevant.

9. 5 servers, local counters, limit 60/min, requests reach all five. Most per minute?
A) 60 B) 120 C) 300 D) 12
Answer C. 5 x 60. A is right only with one shared counter; B and D match no step.

10. Which statement about the framework is true?
A) memorise each famous product's architecture B) it is scaffolding to adapt to the interviewer C) the minutes are strict limits D) always skip estimation
Answer B [S1]; time budgets are soft. A reads as inflexibility; D removes the numbers that justify the design.

FIVE SENTENCES TO KEEP

Recap and the 60-second answer

1. A design round tests how you decide **what** to build and why; the prompt is vague on purpose.
2. First decide the round type: LLD (classes and code) or HLD (components and scale). Ask if unsure.
3. Turn the prompt into a scope box: features in, features out, scale and numeric targets (99.9% is about 8.76 hours of downtime a year).
4. Flow: clarify, estimate, data and API, high-level design (5 to 7 boxes), deep dive and stress test. Chai Every Day Helps Developers.
5. Commit to choices, name the cost, test your own design, and follow the interviewer if they redirect.

SAY THIS IN AN INTERVIEW: "HOW DO YOU APPROACH A SYSTEM DESIGN PROBLEM?"

"First I clarify. I confirm whether you want class design or architecture, list three to five features in scope, say what I am leaving out, and agree numeric targets such as availability and p99 latency. Second, I estimate scale: users, requests per second and data size, with my assumptions stated. Third, I define the main entities and four or five endpoints. Fourth, I draw a high-level design with five to seven components and explain why each is there. Fifth, I go deep on the two riskiest components and stress-test the design: what breaks at ten times the load, what happens if a machine fails, and what it costs. At each decision I state the trade-off I am accepting."

COVERAGE MAP: WHERE EACH IDEA GOES NEXT

SUB-TOPIC	COVERED HERE	WHERE ELSE
What a design round is; LLD vs HLD	Fully	Lesson 15 (LLD method)
FR and NFR; numeric targets	Fully	Lesson 11 (reliability)
Availability maths	Fully (series and parallel)	Lesson 11
Scale estimation	Preview only	Lesson 4
Data and API	Introduced	Lesson 2
High-level design	Introduced with one example	Lessons 24 to 31
Deep dive and stress test	Introduced with one example	Lessons 24 to 31
Communication in the round	Covered	Lesson 33 (mock)

CHECK OUR WORK

Sources

S = opened in full; X = seen only in search-result extracts. Every availability, QPS and storage figure in this lesson was recomputed in code.

ID	PAGE AND WHAT IT SUPPORTED	URL
S1	DesignGurus: The Four-Step System Design Interview Framework (updated May 2026). Framework, time budgets, mistakes, scoping, deep dive, LLD exclusion	designgurus.io/system-design-interview/framework
S2	LeetCode: Amazon SDE 1, full interview loop (undated). SDE-1 LLD round, 5 to 6 follow-ups, "LLD is fair game even for SDE1"	leetcode.com/discuss/post/8293563/amazon-sde-1-full-interview-loop-experie-n3xm/
S3	PracHub: Amazon new-grad experience (Aug 2026; published 13 Sep 2026). Vague rate-limiter prompt, the hook	prachub.com/interview-experiences/amazon-new-grad-software-engineer-interview-experience-aws-offer-after-four-onsite-rounds
S4	LeetCode: Flipkart SDE-1 (posted Mar 2026; loop 2024). No design round; HM round on OOPs, DBMS, CN, OS, projects	leetcode.com/discuss/post/7691468/flipkart-interview-experience-for-sde-1-bgs2k/
S5	LeetCode: Microsoft SDE II (rounds 27 Feb and 2 Mar 2026). The "why" behind data-structure choices	leetcode.com/discuss/post/7620569/microsoft-sde-ii-interview-experience-re-hqua/
S6	LeetCode: Microsoft SDE 2 L61/L62, India, 2026. LLD and HLD rounds; reasoning valued	leetcode.com/discuss/post/7769548/microsoft-interview-experience-sde-2-l61-xdc2/
S7	Glassdoor: Amazon Bengaluru (interview Mar 2025, posted 13 Aug 2025). "Low level design of parking lot"	fr.glassdoor.ca/Entretien/Amazon-Entretien-E6036-RVW99346274.htm
S8	Glassdoor: Amazon San Francisco (May 2025). Parking-system question	fr.glassdoor.ch/Entretien/Amazon-Entretien-E6036-RVW97878809.htm
S9	Glassdoor: Amazon Embedded SDE (Jun 2025). URL shortener as the last of four rounds	glassdoor.co.in/Interview/Amazon-Interview-E6036-RVW102556955.htm
S10	Glassdoor: Cisco Senior SWE reports (incl. Jul 2026). Own-system whiteboard: QPS, rate limiting, circuit breaker, queues	glassdoor.com.br/Entrevista/Cisco-Entrevista-E1425-RVW103576482.htm
S11	TechPrep: Adobe interview process 2026 (aggregator). LLD vs HLD; HLD for MTS-2+	techprep.app/companies/adobe
S12	LeetCode: Atlassian SDE-2 India (undated). Requirements, API, DB, list of services	leetcode.com/discuss/interview-experience/1795183/atlassian-sde-2-remote-india
S13	InterviewBit: System Design interview questions (updated Jan 2024). Beginner list, availability formula, "how to answer"	interviewbit.com/system-design-interview-questions
X1	Blind (search extract): Google L3 reportedly gets no system design	teamblind.com/post/no-sys-design-in-google-onsite-goodbad-qslc0w4t
X2	Taro and TechPrep (search extracts): Uber LLD/machine-coding and HLD reports	jointaro.com/interviews/companies/uber/experiences/sde-intern-bangalore-rural-july-1-2025-accepted-offer-positive-5441773a/ · techprep.app/companies/uber
X3	Alex Xu's four steps, as summarised by others (book not opened)	system-design.space/en/chapter/interview-frameworks/ · business-tips.odoo.com/blog/amazon-book-reviews-hub-1/system-design-interview-an-insiders-guide-5-key-lessons-summary-and-main-idea-14

SELF-CHECK

Can you say yes to all four?

I can tell LLD from HLD
And I ask one question when I am not sure.

I can write a scope box
In, out, scale, and numeric targets.

I can do the nines
99.9% is 8.76 h a year; chains multiply UPs, copies multiply DOWNs.

I can run the 5 steps
Chai Every Day Helps Developers, inside 45 minutes.

Honest gaps: what this lesson only touches or leaves out

- No target company's published rubric was found; the framework comes from one prep site and secondary summaries of Alex Xu's book.
- What was actually graded in the Amazon hook round is unknown; we have the candidate's impression only.
- Fresher-specific design reports are few; most evidence is SDE-2 or senior.
- Uber and Google claims come from search extracts, not opened pages.
- Not covered: frontend system design, AI-adjacent design questions, security and authentication, multi-region design, cost modelling, behavioural rounds. Not claimed: that nothing else can be asked.

> "Design a rate limiter." **Now you ask four questions, write a scope box, and only then pick up the marker.**

YOUR NEXT STEPS

STEP 01
Solve problems 1 to 6 on paper

STEP 02
Say the 60-second answer aloud, twice

STEP 03
Read ahead: Lesson 2, client-server and HTTP